Modern Fortran Software Development Weekly Learning Plan

A 12-week, code-first roadmap for scientific software engineering

Duration	Weekly cadence	Primary outcome
12 weeks	1 topic + 1 code deliverable	A small, tested Fortran project portfolio

How to use this plan: Work one week at a time. At the end of each week, commit your code, write a short note about what you learned, and keep the deliverable in a single Git repository. Use fpm for new projects unless the week explicitly asks you to practice lower-level build steps.

Recommended repository layout for the full 12 weeks

Keep each week in its own folder under a single repo so you can compare styles and refactors over time.

```
modern-fortran-roadmap/
|- README.md
|- notes/
|  |- week01.md
|  |- week02.md
|  `- ...
|- week01_timeline/
|- week02_scientific_basics/
|- week03_kinematics/
|- week04_diffusion1d/
|- week05_mesh_state/
|- week06_run_config_demo/
|- week07_testing/
|- week08_solver_design/
|- week09_c_interop/
|- week10_perf_lab/
|- week11_parallel_lab/
`- week12_capstone/
```

Suggested per-project structure

```
project_name/
|- fpm.toml
|- app/
|  `- main.f90
|- src/
|  `- project_name_*.f90
|- test/
|  `- test_*.f90
|- data/
|  `- sample_input.nml
|- scripts/
|  `- run_debug.sh
```

```
|- README.md
`- notes.md
```

Default compiler habits: debug builds with warnings, runtime checks, and backtraces; release builds with optimization only after correctness is verified. Prefer modules, allocatables, explicit intents, and tests over legacy patterns such as COMMON blocks or implicit typing.

Week 1 - History of Fortran and the modern ecosystem

Week	Core outcome	Deliverable
1	History of Fortran and the modern ecosystem	Program that prints a short timeline and one headline feature per major standard.

Weekly goals

- Understand the major standards: FORTRAN I/66/77, Fortran 90/95, 2003, 2008, 2018, and the direction of 2023.
- Identify what changed from legacy style to modern style: free-form source, modules, allocatables, array syntax, C interoperability, and parallel features.
- Get familiar with the current ecosystem: gfortran, fpm, stdlib, LFortran, and community resources.

Checklist

- ☐ Read a short timeline of the language.
- ☐ Write a one-paragraph note on why old perceptions of Fortran are incomplete today.
- ☐ Install a compiler and verify `gfortran --version` or the equivalent on your system.

Starter code

```
program fortran_timeline
  implicit none
  print *, "1957 - FORTRAN I: first widely used high-level scientific language"
  print *, "1978 - Fortran 77: structured legacy base for many HPC codes"
  print *, "1991 - Fortran 90: modules, array syntax, free-form source"
  print *, "2004 - Fortran 2003: object-oriented features and C interop"
  print *, "2010 - Fortran 2008: coarrays and additional modernization"
  print *, "2018 - Fortran 2018: further parallel and interoperability work"
end program fortran_timeline
```

Week 2 - Toolchain setup and your first modern project

Week	Core outcome	Deliverable
2	Toolchain setup and your first modern project	A minimal fpm package with one module and one passing test.

Weekly goals

- Set up a compiler, editor support, and a repeatable build workflow.
- Learn the structure of an fpm project.
- Practice building, running, and testing from the command line.

Checklist

- ☐ Install fpm.
- ☐ Create your first package with app/, src/, and test/.
- ☐ Add a README with build and run commands.

Starter code

```
module scientific_basics_math
  implicit none
contains
  pure real function square(x)
    real, intent(in) :: x
    square = x*x
  end function square
end module scientific_basics_math

program main
  use scientific_basics_math, only : square
  implicit none
  print *, "square(3.0) = ", square(3.0)
end program main
```

Week 3 - Core language refresh: procedures, intents, kinds, and modules

Week	Core outcome	Deliverable
3	Core language refresh: procedures, intents, kinds, and modules	A kinematics utility with procedures for velocity, acceleration, and kinetic energy.

Weekly goals

- Practice explicit interfaces through modules.
- Use intent(in), intent(out), and intent(inout).
- Begin writing small reusable scientific utilities.

Checklist

- ☐ Create a module with several procedures.
- ☐ Use implicit none everywhere.
- ☐ Write one simple test for each routine.

Starter code

```
module kinematics
  implicit none
```

```
  private
  public :: velocity, acceleration, kinetic_energy
contains
  pure real function velocity(dx, dt)
    real, intent(in) :: dx, dt
    velocity = dx / dt
  end function velocity

  pure real function acceleration(dv, dt)
    real, intent(in) :: dv, dt
    acceleration = dv / dt
  end function acceleration

  pure real function kinetic_energy(mass, speed)
    real, intent(in) :: mass, speed
    kinetic_energy = 0.5*mass*speed**2
  end function kinetic_energy
end module kinematics
```

Week 4 - Arrays, slices, and numerical thinking

Week	Core outcome	Deliverable
4	Arrays, slices, and numerical thinking	A 1D heat equation stepper using both loops and slices.

Weekly goals

- Use assumed-shape arrays and whole-array operations.
- Compare explicit loops with array syntax.
- Understand why array programming is one of Fortran's strongest areas.

Checklist

- ☐ Implement a simple 1D diffusion step.
- ☐ Write both a loop-based and slice-based version.
- ☐ Compare outputs for the same initial condition.

Starter code

```
subroutine step_diffusion(u_old, u_new, alpha)
  implicit none
  real, intent(in)  :: u_old(:), alpha
  real, intent(out) :: u_new(size(u_old))

  u_new = u_old
  u_new(2:size(u_old)-1) = u_old(2:size(u_old)-1) + alpha * &
       (u_old(3:size(u_old)) - 2.0*u_old(2:size(u_old)-1) + u_old(1:size(u_old)-2))
end subroutine step_diffusion
```

Week 5 - Data modeling with allocatables and derived types

Week	Core outcome	Deliverable

5	Data modeling with allocatables and derived types	A derived type representing a 2D scalar field.

Weekly goals

- Use allocatable arrays safely.
- Model simulation state with derived types instead of global variables.
- Prefer allocatable over pointer unless you need pointer semantics.

Checklist

- ☐ Create a field type that stores metadata and data.
- ☐ Add initialization and norm routines.
- ☐ Practice moving state through procedures cleanly.

Starter code

```
module mesh_state
  implicit none
  type :: field2d
     integer :: nx = 0, ny = 0
     real, allocatable :: values(:,:)
  contains
     procedure :: l2_norm
  end type field2d
contains
  real function l2_norm(this)
    class(field2d), intent(in) :: this
    l2_norm = sqrt(sum(this%values**2))
  end function l2_norm
end module mesh_state
```

Week 6 - I/O, configuration, and reproducibility

Week	Core outcome	Deliverable
6	I/O, configuration, and reproducibility	A toy simulation driven by a namelist input file.

Weekly goals

- Read simulation settings from a file rather than hardcoding them.
- Separate I/O from numerical kernels.
- Record enough metadata to reproduce a run.

Checklist

- ☐ Use newunit= for file handles.
- ☐ Read a simple namelist or text config.
- ☐ Write results and a run metadata file.

Starter code

```
module config_io
  implicit none
contains
  subroutine read_config(nx, nsteps, dt, filename)
    integer, intent(out) :: nx, nsteps
    real, intent(out) :: dt
    character(len=*), intent(in) :: filename
    integer :: iu
    namelist /run_cfg/ nx, nsteps, dt
    open(newunit=iu, file=filename, status='old', action='read')
    read(iu, nml=run_cfg)
    close(iu)
  end subroutine read_config
end module config_io
```

Week 7 - Testing scientific code

Week	Core outcome	Deliverable
7	Testing scientific code	A test file that checks one numerical routine with a tolerance.

Weekly goals

- Write deterministic tests for kernels and parsers.
- Use tolerances for floating-point comparisons.
- Treat testing as part of scientific credibility, not an optional extra.

Checklist

- ☐ Add tests for at least three earlier routines.
- ☐ Create one regression output file and compare against it.
- ☐ Document your test philosophy in a short note.

Starter code

```
program test_kinematics
  use kinematics, only : kinetic_energy
  implicit none
  real :: got, expect, tol
  got = kinetic_energy(2.0, 3.0)
  expect = 9.0
  tol = 1.0e-6
  if (abs(got - expect) > tol) error stop "kinetic_energy failed"
  print *, "tests passed"
end program test_kinematics
```

Week 8 - Scientific software design and selective OOP

Week	Core outcome	Deliverable

8	Scientific software design and selective OOP	A solver type that advances a field by one step.

Weekly goals

- Separate model state, solver logic, I/O, and driver code.
- Learn where Fortran OOP helps and where plain modules are simpler.
- Refactor code into a maintainable package layout.

Checklist

- ☐ Create small focused modules.
- ☐ Use a derived type with bound procedures where it improves clarity.
- ☐ Keep numerical kernels testable and side-effect-light.

Starter code

```
type :: solver_type
   real :: alpha = 0.0
 contains
   procedure :: advance
 end type solver_type

subroutine advance(this, u_old, u_new)
  class(solver_type), intent(in) :: this
  real, intent(in)  :: u_old(:)
  real, intent(out) :: u_new(size(u_old))
  call step_diffusion(u_old, u_new, this%alpha)
end subroutine advance
```

Week 9 - Interoperability with C and mixed-language workflows

Week	Core outcome	Deliverable
9	Interoperability with C and mixed-language workflows	A Fortran AXPY routine callable from C.

Weekly goals

- Understand iso_c_binding and bind(C).
- Design a narrow, stable API boundary.
- See how Fortran can serve as the compute core inside a larger workflow.

Checklist

- ☐ Export one routine with C interoperability.
- ☐ Call it from a tiny C driver.
- ☐ Write down memory ownership rules for the interface.

Starter code

```
module c_axpy_mod
  use iso_c_binding
  implicit none
contains
  subroutine c_axpy(n, a, x, y) bind(C, name="c_axpy")
    integer(c_int), value :: n
    real(c_double), value :: a
    real(c_double), intent(in) :: x(n)
    real(c_double), intent(inout) :: y(n)
    y = a*x + y
  end subroutine c_axpy
end module c_axpy_mod
```

Week 10 - Performance engineering and profiling mindset

Week	Core outcome	Deliverable
10	Performance engineering and profiling mindset	A small benchmark harness comparing kernel variants.

Weekly goals

- Benchmark before rewriting.
- Understand the trade-offs among loops, array syntax, and do concurrent.
- Track correctness while optimizing.

Checklist

- ☐ Implement one kernel three ways.
- ☐ Time them on the same input size.
- ☐ Record observations about temporaries, cache behavior, and compiler options.

Starter code

```
do concurrent (i = 2:n-1)
  u_new(i) = u_old(i) + alpha*(u_old(i+1) - 2.0*u_old(i) + u_old(i-1))
end do
```

Week 11 - Parallel Fortran: OpenMP, MPI awareness, and coarrays

Week	Core outcome	Deliverable
11	Parallel Fortran: OpenMP, MPI awareness, and coarrays	An OpenMP parallel loop for a matrix or vector kernel.

Weekly goals

- Learn the difference between shared-memory and distributed-memory parallelism.
- Parallelize a simple loop with OpenMP or explore a tiny coarray example.

- Measure scaling or at least correctness under parallel execution.

Checklist

- ☐ Choose one path: OpenMP or coarrays.
- ☐ Start from a serial code you already trust.
- ☐ Write a short summary of what changed and what stayed hard.

Starter code

```
!$omp parallel do default(shared) private(i)
do i = 1, n
  y(i) = a*x(i) + y(i)
end do
!$omp end parallel do
```

Week 12 - Capstone: a small scientific software package

Week	Core outcome	Deliverable
12	Capstone: a small scientific software package	A Git-ready capstone package with README, tests, data, and a brief design note.

Weekly goals

- Assemble the best ideas from the prior weeks into a coherent mini-project.
- Include project structure, tests, inputs, documentation, and at least one advanced feature.
- Write a short engineering note explaining why Fortran was a good fit.

Checklist

- ☐ Pick a realistic toy problem: diffusion, Poisson, random walk transport, or another compact solver.
- ☐ Use fpm, tests, config-file input, and documented runs.
- ☐ Include either interop or parallelism.
- ☐ Publish the repository cleanly.

Starter code

```
program capstone_main
  use solver_driver_mod, only : run_case
  implicit none
  call run_case("data/case01.nml")
end program capstone_main
```

Language fit guide: when Fortran is strong and when to choose something else

Use Fortran when…	Watch-outs	Prefer another language when…

You are writing array-heavy numerical kernels or CPU-centric HPC routines.	Ecosystem and package availability are smaller than Python or C++.	You need rapid scripting, plotting, notebooks, ML workflows, or broad data tooling - use Python.
You must maintain or modernize legacy scientific codes already written in Fortran.	Tooling quality varies by compiler and editor setup.	You need rich application architecture, large general-purpose libraries, or complex GUIs - use C++, Rust, Java, or Go.
You want clean mathematical expression of finite-difference, linear algebra, or simulation loops.	Some teams still use legacy habits, so code review standards matter.	You need memory-safe systems programming or infrastructure services - use Rust.
You can pair Fortran with Python or C at the workflow boundary.	GPU-first workflows may be more natural in CUDA/HIP/C++ stacks.	You want an interactive technical language with a broad scientific ecosystem and JIT-oriented workflows - consider Julia.

External references and learning resources

Fortran-lang Learn and Best Practices: Official learning hub with quickstart material, modules/programs guidance, allocatable arrays, arrays, and style advice.

fpm documentation: Official documentation for the Fortran Package Manager, including package layout, commands, and tutorials.

Fortran stdlib: Community standard library for utilities that are not part of the language standard.

GNU Fortran manual: Compiler reference for options, diagnostics, interoperability, and GNU-specific behavior.

Modern Fortran Explained: Deep reference text covering modern language features and the standard.

FortranCon videos: Community talks on modern practice, tooling, and scientific workflows.

Official URLs: fortran-lang.org/learn | fortran-lang.org/learn/best_practices | fpm.fortran-lang.org | stdlib.fortran-lang.org | gcc.gnu.org/onlinedocs/gfortran.pdf | global.oup.com/.../modern-fortran-explained-9780198876588 | youtube.com/@fortrancon/videos

Suggested weekly rhythm

- Day 1: read the core concept and study the starter code.
- Day 2: type the code yourself and make it run from the command line.
- Day 3: modify the example and add one extra feature.
- Day 4: add at least one test and write a short learning note.
- Day 5: clean the repository, commit the work, and summarize the main takeaway.

End-of-plan output

By week 12 you should have: a clean portfolio repo, repeated practice with fpm, modules, tests, arrays, allocatables, interop, and performance thinking, plus a realistic sense of where Fortran fits in a modern scientific software stack.